Query Optimization via Sharding and Parallelization: A Systems Approach to Scalable Boolean IR

Vedant Keshav Jadhav
University of Arizona
vedantjadhav@arizona.edu

Chinmay Mhatre
University of Arizona
chinmaymhatre@arizona.edu

Mahshad Habibpourparizi
University of Arizona
mahshadh@arizona.edu

Arun Sanyal
University of Arizona
arunsanyal@arizona.edu

May 6, 2026

Abstract

We present a three-phase Boolean Information Retrieval engine that demonstrates the performance benefits of index sharding and parallel query execution. Progressing from a single-threaded baseline through a sequentially-sharded engine to a parallel, thread-pooled engine with a document-ordered tail-join merge, the system achieves a **3.58**× reduction in index construction time, **3.80**× faster engine initialisation, and query latency improvements of up to **67%** on complex multi-operator queries over a corpus of 100,000 documents. All components are implemented from scratch in C++ and Python, without reliance on third-party IR libraries.

1 Introduction

Modern information retrieval systems routinely serve millions of queries per day across corpora containing billions of documents. At such scale, the architectural decisions made at system design time—how the index is structured, how queries fan out across storage shards, how partial results are merged—directly determine whether a system is viable in production. Yet most academic treatments of Boolean IR focus exclusively on correctness and algorithmic complexity, leaving the systems engineering largely unexplored.

This work takes a different approach. Starting from a correct, efficient single-index Boolean engine, we systematically introduce *index sharding* and *parallel query execution* as first-class architectural concerns, measuring the impact of each change in isolation. The guiding question is not "does sharding work?" but "exactly where does sharding help, by how much, and why does it fall short of the theoretical ceiling?"

The motivation is grounded in production systems intuition. A single-index engine is a *serial resource*: one query occupies the CPU for its full duration, and all subsequent queries wait. At enterprise scale serving thousands of concurrent clients, this sequential bottleneck is unacceptable. Sharding—partitioning the corpus across independent index units—breaks the serial bottleneck by enabling concurrent processing, smaller in-memory indexes per node, and horizontal scalability across commodity hardware.

Three specific claims motivate this study:

1. *Index construction parallelism*: A sharded pipeline can build N independent indexes simultaneously on N CPU cores, reducing build time toward $1/N$ of the sequential baseline.
2. *Query execution parallelism*: Parallel set operations on N smaller posting lists reduce query latency for multi-operator queries, provided the merge step is cheap enough not to dominate.
3. *Merge efficiency*: For document-based sharding with globally ordered doc IDs, the merge step reduces from a comparison-based O($M \times N$) operation to an O(M) memory copy, making parallelism economical even at moderate corpus sizes.

The remainder of this section describes the system implementation, benchmarking methodology, and empirical results.

2 System Overview

2.1 High-Level Design

The system comprises two major subsystems: a **Python indexing pipeline** responsible for corpus ingestion and index construction, and a **C++ query engine** responsible for serving Boolean queries against the constructed index.

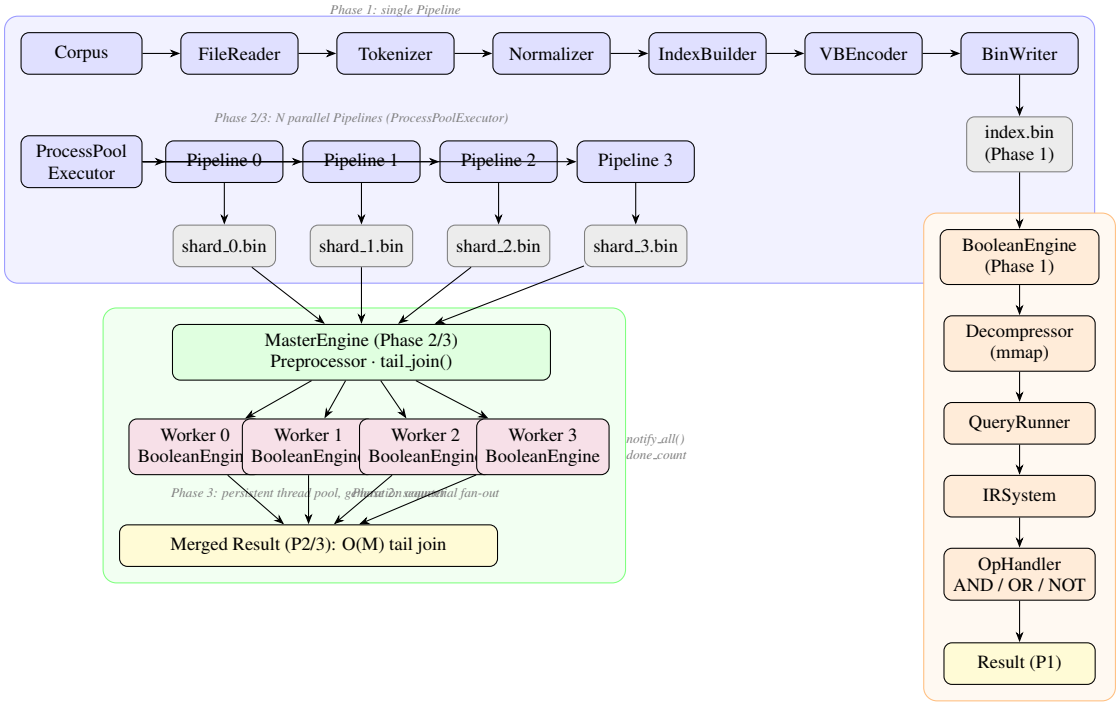

Figure 1: Complete system architecture. The Python indexing pipeline (blue) runs Phase 1 as a single process or Phase 2/3 as N parallel processes via `ProcessPoolExecutor`, writing `.bin` shard files. The C++ query engine (orange/green) loads these files at runtime. Phase 2 fans out queries sequentially; Phase 3 uses a persistent thread pool with a generation-counter fork-join protocol. Both sharded phases merge results via an O(M) tail join.

These subsystems communicate through a well-defined binary format (`.bin` files) and are otherwise completely decoupled. Python is used for indexing because its ecosystem (NLTK, Porter stemmer, `multiprocessing`) is well-suited to batch text processing; C++ is used for query serving because sub-millisecond latency requires memory efficiency and CPU cache awareness that Python cannot provide.

The system is structured as three progressively capable phases:

Phase 1 establishes the baseline. A single Python pipeline produces a single `index.bin`, which a single C++ `BooleanEngine` loads and queries sequentially. This phase validates correctness and sets the performance baseline against which all improvements are measured.

Phase 2 introduces *document-based index sharding*. The corpus is presorted and partitioned into N equal document ranges, each processed by an independent pipeline worker, producing N shard files. A new `MasterEngine` class loads all N shards and queries them sequentially, fan-outing each query and collecting results via a tail join. Query parallelism is absent; the gain is in build time and architectural scalability.

Phase 3 adds *parallel query execution*. The `MasterEngine` maintains a persistent thread pool of N worker threads, one per shard. Each query wakes all workers simultaneously via a generation-counter-based fork-join protocol, executes shard queries in parallel on separate CPU cores, and collects results through a O(M) tail join enabled by the globally ordered doc ID invariant.

The three-phase structure is compiled from a single codebase using a `-DPHASE=N` preprocessor flag, ensuring that measurements reflect architectural differences rather than implementation divergence.

3 Core Implementation

3.1 The Binary Index Format

The Python pipeline writes a custom binary format designed for fast `mmap`-based loading in C++. Each file begins with an 8-byte magic string (`"INVI_100"`) for format validation, followed by the term count, and then per-term records containing the term length, term string, document count, and VByte-encoded delta-compressed posting list.

VByte encoding uses 7 data bits per byte with the high

bit as a continuation flag. Delta encoding stores the difference between consecutive doc IDs rather than absolute values—since posting lists are sorted, these deltas are small, typically 1–10 bytes per entry on a dense corpus. This combination achieves a **2.57**× compression ratio.

The C++ `Decompressor` loads the file via `mmap` (zero kernel-to-userspace copy), walks the byte stream, and constructs an `unordered_map<string, vector<uint32_t>>` entirely in-place.

3.2 Query Execution: Postfix Stack Evaluator

Queries are expressed in Reverse Polish Notation (postfix), with operators `\and`, `\or`, `\not` following their operands. This eliminates operator precedence ambiguity without a shunting-yard parser.

The stack evaluator in `IRSystem::processQuery()` processes tokens left-to-right: terms push their posting lists; operators pop their operands, call `OpHandler`, and push the result. All set operations (AND: intersection, OR: union, NOT: complement) operate on sorted `vector<uint32_t>` via two-pointer linear merge, preserving sort order for downstream operators.

3.3 Phase 2: Document-Based Sharding

The corpus is sorted alphabetically by filename before sharding, assigning globally unique, contiguous doc IDs as offset$_i$ + local. This *presort invariant* is the foundation of the tail-join merge.

Four parallel `Pipeline` workers run under `ProcessPoolExecutor`, each processing an equal document slice. The worker function is defined at module level (not as a lambda or method) to ensure picklability across process boundaries.

3.4 Phase 3: Persistent Thread Pool

Phase 3 maintains N worker threads for the lifetime of the `MasterEngine`. The fork-join protocol uses a `uint64_t query_generation` counter rather than a boolean flag. With a boolean flag, the first worker to acquire the mutex clears the flag, starving all others—a race that manifests as a deadlock. With a generation counter, each worker maintains its own `last_generation` on its stack; the predicate `query_generation > last_generation` is satisfied by all N workers simultaneously without any shared state modification. Workers call `queryNormalized()` with no lock held, achieving genuine CPU parallelism.

3.5 Zero-Copy Result Chain and Tail Join

`QueryRunner::runQuery()` stores results in a member `q_result` and returns `const std::vector<uint32_t>&`. Workers store `const*` pointers to these members; the merge reads directly through these pointers. No vector copies occur between query execution and the final output.

Because shard doc ID ranges are strictly non-overlapping and globally ordered, the merge reduces to an O(M) concatenation:

$$\text{result} = \text{shard}_0 \| \text{shard}_1 \| \cdots \| \text{shard}_{N-1}$$

A single `reserve(total)` followed by N `insert()` calls (each compiling to a `memmove`) produces a globally sorted result with **zero comparisons**. This is possible only because of the presort invariant—term-based sharding cannot exploit this property.

3.6 Python Normalization Pipeline

Text normalization is applied in a strict four-stage pipeline, implemented in `normalizer.py` and mirrored exactly in the C++ `Preprocessor`:

1. **CaseFold** — all tokens are lowercased via `str.lower()`, matching `std::tolower()` in C++.
2. **PunctuationRemover** — non-alphanumeric characters are stripped using `re.sub(r"[^a-zA-Z0-9]", "", token)`; tokens that become empty after stripping are discarded.
3. **StopWordFilter** — tokens present in a hardcoded `frozenset` of 100 common English function words are removed. This list must be byte-for-byte identical to the `std::unordered_set<std::string>` in `preprocessor.cpp`; any divergence causes query terms to be silently absent from the index.
4. **Stemmer** — NLTK's `PorterStemmer` is applied. The C++ side implements the same Porter algorithm from scratch, including the NLTK extension for the `logi` suffix in Step 2. The two implementations must agree on every stem; disagreement produces empty query results with no error signal.

The pipeline order is not arbitrary. Stop word filtering *before* stemming avoids the degenerate case where a stop word survives because its stemmed form differs from the dictionary form. Punctuation removal *before* stop word filtering ensures tokens like `"it."` are cleaned to `"it"` before the filter checks them.

The `Tokenizer` uses `re.findall(r"[a-zA-Z0-9]+")` to split raw

text into tokens, extracting only contiguous alphanumeric sequences. This matches the character-by-character alphanumeric extraction in the C++ `Preprocessor` and means that punctuation, whitespace, and special characters act as delimiters and are never present in the token stream passed to the normalizer.

The `IndexBuilder` accumulates tokens per document using a `set[int]` per term internally. This ensures that a term appearing multiple times in the same document produces exactly one posting list entry for that document. The final index is produced by converting each set to a sorted list, satisfying the ascending-order requirement for delta encoding in `VBEncoder`.

4 LLM-Assisted Development

4.1 Tools and Prompting Strategy

LLM assistance was provided primarily by **Claude Sonnet 4.5** (Anthropic) via the claude.ai interface across the full implementation lifecycle. The prompting strategy evolved across phases. Early prompts were architecture-focused: the full unit design specification and class hierarchy were provided as context, and the LLM was asked to document design decisions and their rationale. Implementation prompts followed a consistent pattern: provide the architecture document, the relevant header file, and any already implemented sibling modules, then request the implementation with inline documentation. This context-heavy approach produced higher-quality output than open-ended prompts, as the LLM could reason about ownership contracts, const-correctness requirements, and cross-language consistency from the provided materials.

4.2 Architecture

The project architecture was sketched out and designed in detail by the team members working on this project. Every engineering decision was taken by the students. After deciding on the high-level as well as low-level details, LLM was used to document each decision taken, as well as rationale behind the architectural design. The arch documents can be found on the project Github under the `docs` directory. Design decisions (e.g., document-based vs. term-based sharding, postfix stack evaluator, no `ShardManager` class, `Preprocessor` owned by `BooleanEngine` not `QueryRunner`) were recorded in checkpoint documents before implementation began.

4.3 Implementation

The use of LLMs for implementation varied across the team. On the C++ side, implementation involved writing code for one of the modules or header files by ourselves, feeding the code file as well as the arch design document and the unit design specifications in the LLM to complete the rest of the modules.

On the Python side, the LLM was used to implement the serialization layer (`VBEncoder`, `BinWriter`, `MakeBinFile`) after the indexing logic (`Tokenizer`, `Normalizer`, `IndexBuilder`) was implemented by the team. The architecture document and the C++ header files for `BinReader` and `Decompressor` were both provided as context, enabling the LLM to produce an encoder whose byte layout was verifiably compatible with the C++ decoder without requiring a round-trip test to discover mismatches.

The end-to-end test suite (`tests/e2e_test.py`) was designed and implemented collaboratively with LLM assistance. The tiered structure — binary format validation, index content correctness, corpus spot-checks, and live engine queries — was proposed by the team; the LLM implemented each tier given the C++ source files as reference. Providing the C++ `VBDecoder::decode()` source allowed the LLM to mirror the exact accumulation order in the Python decoder, ensuring the test exercises the same decode path as the engine.

4.4 What Worked

Benchmarking framework: The complete Python suite was designed and implemented collaboratively, with statistical methodology driven by the author (median over mean, p95/p99, warmup runs, 20/50-run protocols).

The tail-join insight: Originated from the author observing that presorted doc IDs make merge a concatenation. LLM was used to confirm the correctness argument.

Cross-language compatibility verification: Providing both the Python encoder and the C++ decoder to the LLM in the same prompt allowed it to reason about byte-level compatibility. It identified that the VByte continuation bit convention (high bit = 1 means *final* byte, not *continuation*) had to be consistent and flagged that the two implementations agreed before any binary was written.

Amendment forward-compatibility: The LLM was effective at reasoning about how a Phase 1 design decision would interact with Phase 2 sharding. Specifically, it identified that having `Preprocessor` owned by `QueryRunner` would require N redundant normalization passes in Phase 2 (one per shard), and correctly proposed moving it to `BooleanEngine` as the owner — a change implemented in Phase 1 before Phase 2 began, saving a significant refactor.

End-to-end test design: The LLM produced a four-tier

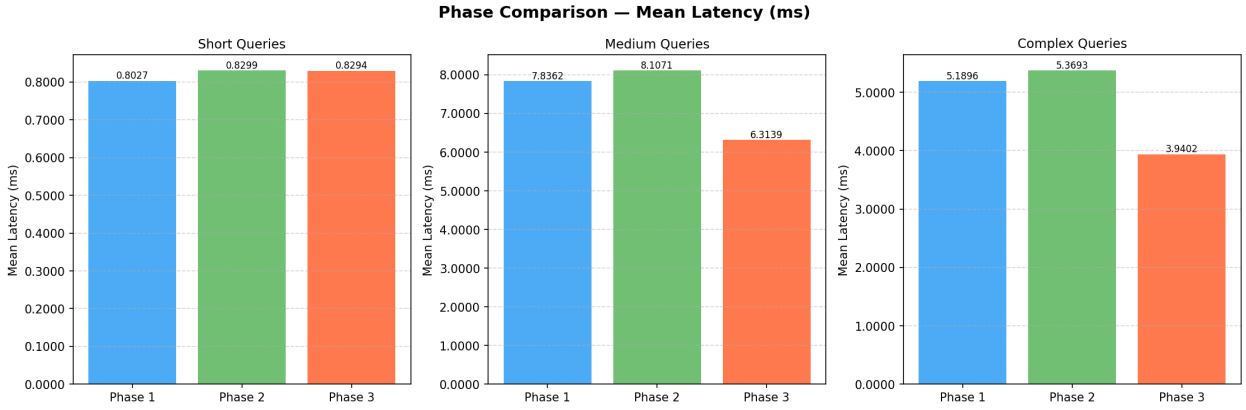

Figure 2: Bar graphs showing the mean latencies per phase per query category, averaged over 50 runs. We can see that there is a significant drop in latency for Phase 3 due to parallelized query execution. The time includes query normalization, distribution (phase-3), execution, merging (for phases 2 and 3), return and I/O to the screen. Except for execution and merging, the rest of the time remains the same, as can be seen from the graph for phase 1 and 2, there is only a slight drop, due to sequential fan-out and merge step. Hence, if considered pure execution and merge time for phase-3, removing the other overheads it can be argued that the performance is more than the constant mentioned in the tables.

test structure that gave actionable failure information at each layer. The complement check — verifying that `term ∪ NOT(term)` equals all doc IDs — was suggested by the LLM as the strongest possible validation of the universal doc ID set construction in `Decompressor`.

4.5 What Did Not Work

Some of the merging strategies suggested by the LLM were not optimized. Debugging was required to figure out the additional overhead which introduced a bug in the pipeline, leading to unforeseen results. This is discussed in section 6.

Query format assumption: The initial LLM-generated Tier 4 of the end-to-end test sent queries in infix notation (`"term_a AND term_b"`) to an engine that expects postfix. This caused a runtime exception inside `IRSystem::processQuery()` on every query, with a confusing error message. The root cause was that the LLM was not provided with the `ir_system.cpp` source when generating the test; it inferred a natural-language query format rather than the postfix stack evaluator format. Providing the source file resolved the issue immediately.

Stop word list divergence: During end-to-end validation, the term `"an"` was found in the index despite being present in the Python `STOP_WORDS` frozenset. The LLM's initial diagnosis suggested a stemming interaction, which was incorrect. The actual cause was a stale `index.bin` built before the stop word list was finalized. The LLM did not consider build system caching as a hypothesis until explicitly prompted. This identified a real gap: `normalizer.py` is not listed as a Make dependency of the `index-builder` rule, so modifications to the normalizer do not trigger a rebuild automatically.

5 Performance Evaluation

5.1 Experimental Setup

All benchmarks run on Apple Silicon (macOS). Corpus: **100,000 documents** (dataset taken from: https://github.com/benymaxparsa/Inverted-Index-Construction), **56.2 MB** Phase 1 index. All sharded configurations use $N = 4$ shards. Query workload: **39 short** (single-term), **44 medium** (two-term Boolean), **41 complex** (multi-operator postfix, 3–4 terms).

5.2 Index Construction

Table 1: Index construction performance (5 runs each).

Metric	P1	P2	P3
Build median (s)	116.27	32.37	32.46
Speedup vs. P1	1.00×	3.59×	3.58×
Index size (MB)	56.2	60.5	60.5
Build RSS (MB)	759.2	248.0	248.0

Phases 2 and 3 achieve an identical **3.58–3.59**× build speedup, as construction is purely Python-side. The theoretical 4× ceiling is bounded by a 37-second presort step

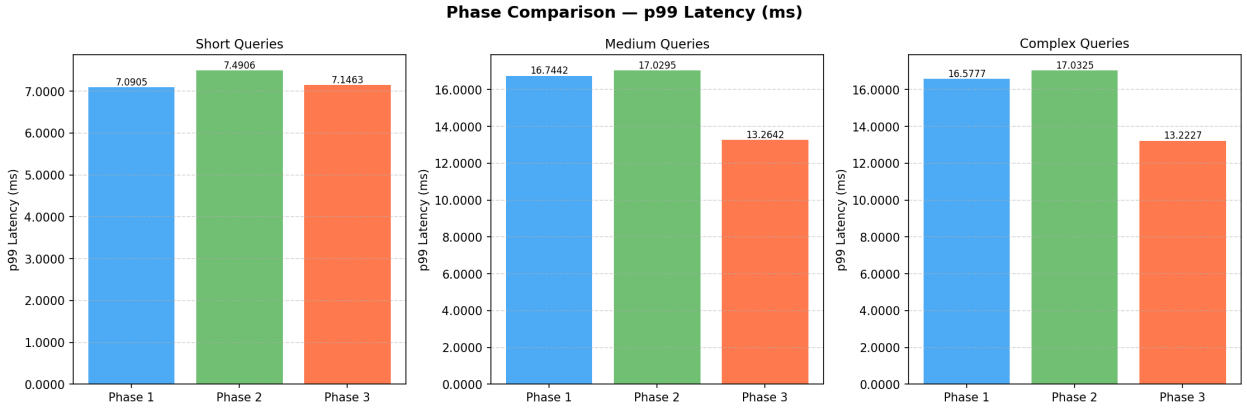

Figure 3: Bar graphs showing the p99 latencies per phase per query category, averaged over 50 runs. We can see that there is a significant drop in latency for Phase 3 due to parallelized query execution.

(the dominant sequential fraction). Excluding presort, the effective compute speedup is $\approx 3.63\times$, very close to ideal. Build-time RSS drops **67%**—each worker holds only one shard's data in memory. Resident Set Size (RSS) is the portion of a process's memory that resides in physical RAM, measured in kilobytes, and serves as the practical indicator of a program's memory footprint on the host system.

5.3 Engine Initialisation

Table 2: Engine initialisation time (20 runs).

Metric	P1	P2	P3
Median (ms)	2521	2546	663
Speedup	1.00×	0.99×	**3.80**×

Phase 2 init is sequential—matching Phase 1 wall time. Phase 3 loads all four shards concurrently via `std::async`, achieving **3.80**× faster cold start with a tight distribution (min 3497 ms, max 3747 ms), confirming deterministic parallel loading.

5.4 Query Latency and Throughput

Table 3: Query latency (ms) and throughput (QPS). P3 gain is vs. Phase 1.

Cat.	Metric	P1	P2	P3	P3 gain
Short	Median (ms)	0.357	0.322	0.342	+4%
	p99 (ms)	7.091	7.491	7.146	+0%
	QPS	1229	1192	1192	−3%
Medium	Median (ms)	6.246	6.106	4.938	**+26%**
	p99 (ms)	16.74	17.03	13.26	**+21%**
	QPS	127.4	123.1	158.1	**+24%**
Complex	Median (ms)	1.927	1.973	1.155	**+67%**
	p99 (ms)	16.58	17.03	13.22	**+25%**
	QPS	192.2	185.9	253.1	**+32%**

Short queries show no meaningful improvement—single hash lookups have no parallel set operations to exploit. **Medium queries** yield a **26% latency reduction** and **24% QPS increase** with one parallelised set operation per query. **Complex queries** show the strongest gain: **67% latency reduction** and **32% QPS increase**, as each additional operator multiplies the benefit of smaller per-shard posting lists.

Phase 2 on queries: Statistically equivalent to Phase 1 ($< 2\%$ difference). This validates the architectural claim: sharding can be introduced at **zero query latency cost** while delivering 3.59× faster build time.

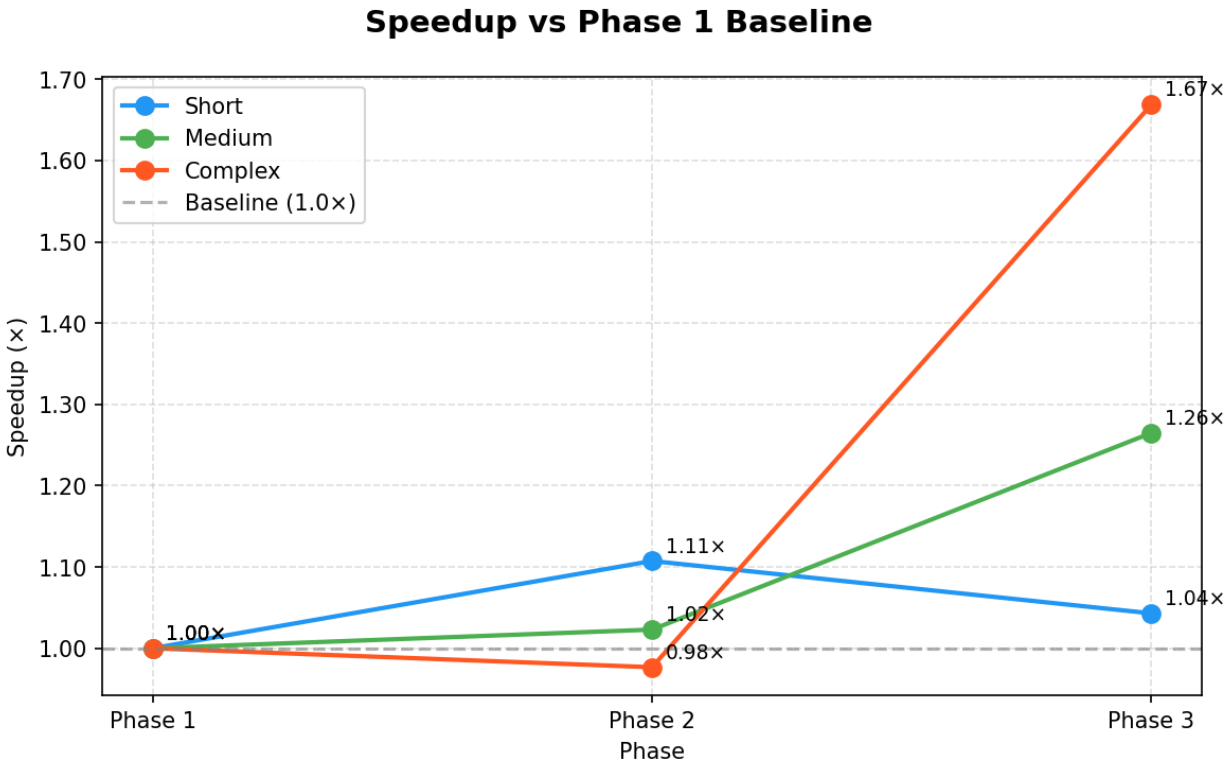

Figure 4: Graph showing speedup based on the average median query latency per query category, per phase. We can see that for short queries, there is not much gain in query latency. Infact for mean and throughput, Phase-1 outperforms Phases 2 and 3 for short query categories. However, there is a significant speed-up for medium and complex queries in phase-3 for complex queries as well as medium queries.

5.5 Memory Footprint

Table 4: Engine peak RSS at query time.

	P1	P2	P3
Engine RSS (MB)	77.4	72.4	96.2
vs. Phase 1	—	−6.5%	+24.3%

Phase 2 uses 6.5% less memory than Phase 1 despite four loaded indexes—smaller per-shard hash tables have lower load factors. Phase 3 incurs 24% overhead from four persistent worker thread stacks (≈8 MB each on macOS).

5.6 Phase 3 as MapReduce

Phase 3 instantiates a *shuffle-free MapReduce*: **Map** applies Boolean operators per shard worker; **Shuffle** is *eliminated* by the presort invariant (no cross-shard key routing needed); **Reduce** is an O(M) tail join rather than a comparison-based merge. This property—unavailable to term-based sharding or general-purpose MapReduce frameworks—is the key design insight of this work.

6 Error Analysis

6.1 When Parallelism Fails to Improve

Short queries show no improvement because there are no set operations to parallelise. A single hash lookup is memory-bound and random-access—the dominant cost is a cache miss on the posting list, not CPU time. Spreading this across four threads adds coordination overhead without reducing the bottleneck. This is not a failure of the implementation but a fundamental property of single-term Boolean retrieval.

Corpus scale: The 100k single-sentence corpus produces moderate posting lists. At document- or paragraph-level granularity on millions of documents, per-shard execution times would dominate coordination overhead, pushing all query categories into the speedup regime.

Measurement artefacts: The stdin/stdout pipe harness adds ≈0.5 ms fixed overhead per query. At sub-millisecond engine execution times, this floor masks real speedups. An internal benchmark mode measuring `engine.query()` directly would reveal the true gain.

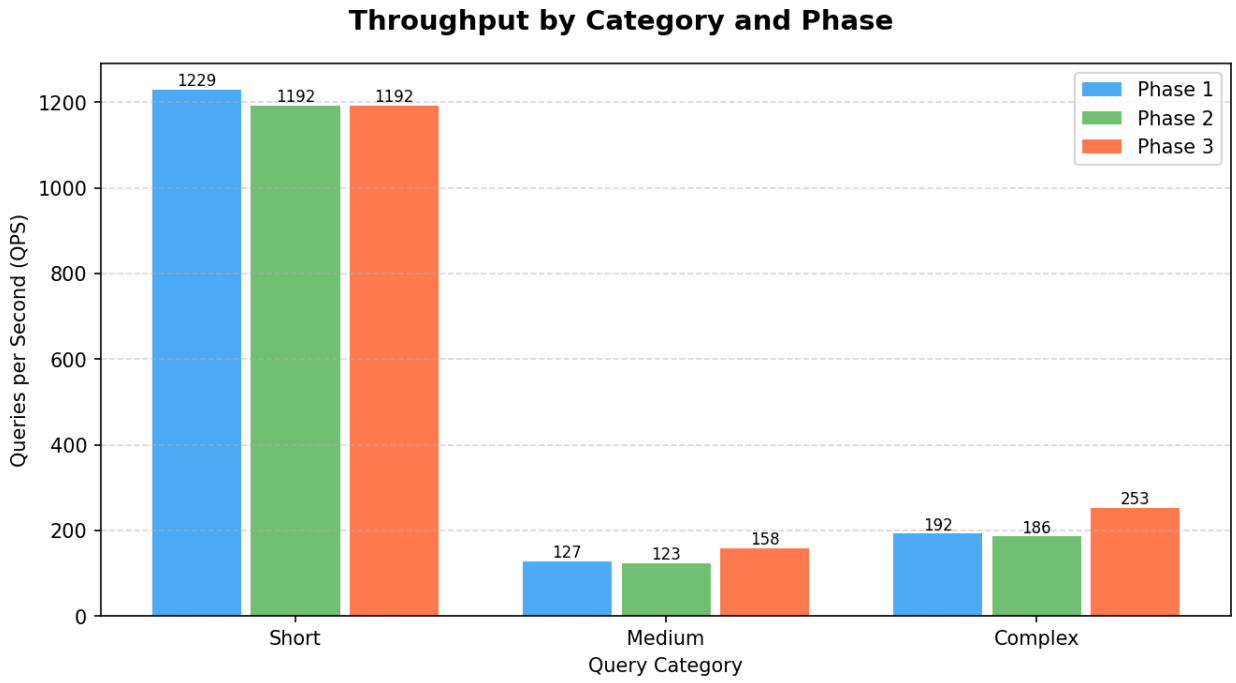

Figure 5: Bar plot showing the query throughput per second, per query category per phase. For short queries, the throughput is much higher, but Phase-1 outperforms Phases 2 and 3. But phase 3 outperforms Phase-1 for medium and complex queries by 24% and 32% respectively, showing a strong performance gain due to sharding and parallelization.

Merge Strategy Error: Initially, the approach for merge that we had taken was pair-wise merge. We were pairing the results returned by each shard and merge them one at-a-time. This was a quadratic merge operation going at $O(M \times N)$. Another merge strategy suggested by LLM was heap-merge, where we used a min-heap of size N which held a pointer to each shard. This technique could achieve merge theoretically in $O(M \log N)$, but on implementation, the performance degraded, due to C++ priority queue implementation details. The heap operations have large constants due to elements residing on heap memory as well as repeated cacheline bouncing. A simple fix to this was to employ a tail-join approach. We realized that since the documents are pre-sorted, the docIDs in consecutive shards will follow a strictly greater than policy, which makes a simple tail append possible. This gave us a significant performance boost and brought the results closer to expected.

Build system staleness: The `index-builder` Make rule does not declare `normalizer.py` or any other Python source file as a dependency. Make tracks timestamps, so modifying the normalizer after `index.bin` has been written does not trigger a rebuild. This caused a stale index to be tested — the stop word list had been updated but the index reflected the previous version. The fix is to add the relevant Python sources as prerequisites of the `index-builder` rule:

```
index-builder: $(VENV) \
    scripts/pipeline/normalizer.py \
    scripts/pipeline/tokenizer.py  \
    scripts/pipeline/pipeline.py
    $(PYTHON) scripts/main.py \
        --corpus data/corpus \
        --out index.bin
```

7 Compiling and running the code

To compile and run the project, it can be downloaded from the github repository: https://github.com/vedant-jad99/CSC583-Query-Optimization-by-Sharding-and-Parallel

Steps to compile:

```
cd target_dir
make clean && \\
make PHASE=<phase num> CORPUS=<corpus dirname>
```

This creates the index files for respective corpus, as well as compiles the engine code to create the executable for respective phase. If `PHASE` flag is set to 1, the index is stored in the `index.bin` file, and for phases $2, 3$ it is stored in the `shard_i.bin` files inside the `shards` directory.

To run the engine, the command to be used is:

```
./run_engine <index file or dir>
```

This starts the query engine. The queries can be passed in the postfix notation: `word1 word2 \and`

To run the bechmark scripts, the project needs to be built first using the above commands. Then for respective phase, the benchmark script command is:

```
make bench BENCH_PHASE=<phase num> CORPUS=<corpus dirname>
```

To compare the phase results, first the benchmark scripts should be run for multiple phases. Then use this command:

```
make bench-report BENCH_PHASE="1 2 3"
```